



Université Saad Dahlab de Blida
Faculté des sciences
Département de mathématiques



Programmation Avancée pour le Web

3^{ème} Année Licence

JavaScript



JavaScript

Concepts et principes fondamentaux

BENAISSI Sellami

Hayat.daoud@gmail.com

2024-2025

PLAN

1. Qu'est ce que Javascript?
2. Que peut-on faire avec JavaScript ?
3. Où met-on les codes JavaScript?
4. Comment afficher les données en JavaScript?
5. Boîtes de messages
6. La Syntaxe de JavaScript .
7. Les variables en JavaScript.
8. Les types de données en JavaScript
9. Les opérateurs dans JavaScript.
10. Les fonctions dans JavaScript ?
11. Les instructions conditionnelles
12. Les boucles dans JavaScript .
13. Les objets dans JavaScript .
14. Les événements.

1

Concepts et définitions

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Qu'est ce que JavaScript ?

Définition :



JavaScript est un langage de programmation de scripts du Web **côté client, orienté objet** à prototype. Il a été inventé en 1995 par **Brendan Eich**, qui travaillait à l'époque pour la société **Netscape**, les programmes ou instructions JavaScript sont intégrés dans les fichiers **HTML** et lus puis appliqués par le navigateur au fur et à mesure.



JavaScript peut aussi s'utiliser comme un langage **côté serveur**, par exemple dans le populaire environnement **Node.js**

Javascript est actuellement développé par Mozilla Corporation.

- | | |
|--|---|
| 1. Qu'est ce que Javascript? | 8. Les types de données en JavaScript |
| 2. Que peut-on faire avec JavaScript ? | 9. Les opérateurs dans JavaScript. |
| 3. Où met-on les codes JavaScript? | 10. Les fonctions dans JavaScript . |
| 4. Comment afficher les données ? | 11. Les instructions conditionnelles |
| 5. Boîtes de messages | 12. Les boucles dans JavaScript . |
| 6. La Syntaxe de JavaScript . | 13. Les objets dans JavaScript . |
| 7. Les variables en JavaScript. | 14. Les événements. |

Que peut-on faire avec JavaScript ?

Avec JavaScript on peut :

- changer le contenu et les attributs HTML
 - modifier les styles HTML (CSS)
 - masquer / afficher les éléments HTML
-
- Effectuer des calculs à la volée
 - Créer des messages utilisateur (alertes ou dialogues)
 - Lire les données d'une table
 - Personnaliser les pages Web à l'exécution
 - Gagnez du temps en éliminant les références de serveur
 - Manipulations de formulaire
 - Jeux par navigateur
 - Formulaire interactif
 - Auto complétion
 - Gérer précisément les événements (clic, survol, touche du clavier...)

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Que peut-on faire avec JavaScript ?

Voici quelques exemples de cas où JavaScript est efficace (voire irremplaçable)

:

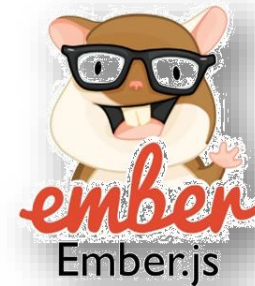
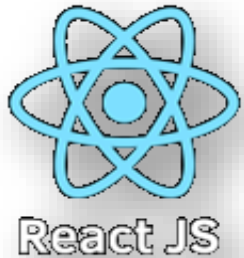
- ☐ **Vérification de saisie dans des formulaires** (*Adresse, email*)
- ☐ **Calculs simples suite à des saisies de formulaire** (*Tarifs, calculatrice*)
- ☐ **Moteur de recherche** (*base de recherche définie par le webmaster*)
- ☐ **Gestion des dates et des heures** (*Date du jour, Date de mise à jour, Calendriers*)
- ☐ **Gestion des cookies** (*Sauvegarde d'information : nombre de visites, caddie virtuel*)
- ☐ **Gestion de la navigation** (*Menus dynamique, popup*)
- ☐ **Animations graphiques** (*Dynamic HTML, MouseOver, bannières rotatives, jeux*)
- ☐ **Création d'application web avec les appels Ajax vers le serveur**
- ☐ **Création de jeux**

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Frameworks JavaScript

samples

Front-End JavaScript Frameworks



Back-End JavaScript Frameworks



1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Bibliothèques JavaScript samples



1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Java vs JavaScript ?

JavaScript



Il ne faut pas confondre **javascript** avec **java**. Ce sont deux langages différents à des finalités différentes aussi



Ce sont deux langages différents.. et qui ne sont pas utilisés dans le même environnement ni pour faire les mêmes choses...

- **JavaScript** ne permet pas de créer des applets ou des applications autonomes.
- **Java** permet de créer des applications qui sont exécutées sur une **machine** virtuel tandis que le code **JavaScript** est exécuté uniquement sur un **navigateur**
- Le code **Java** doit être **compilé** tandis que le code **JavaScript** est composé entièrement **de texte**.

2

Les bases

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Où met-on les codes JavaScript?

Le code JavaScript peut se présenter de deux manières :

- intégré dans une page HTML ou
- intégré dans un fichier externe.

Où met-on les codes JavaScript?

- Vous pouvez placer JavaScripts dans :

- 1 • l'en-tête (<head>),
- 2 • le corps (<body>),

Vous pouvez le placer n'importe où



alert(' Salam ');

```
1 <!doctype html>
2 <html>
3 <head>
4   <title> Salam World </title>
5   <script> alert(' Salam ');</script> 1
6 </head>
7 <body>
8   <script> alert(' Salam ');</script> 2
9   <script src="myScript.js"></script> 3
10 </body>
11 </html>
```



myScript.js

alert(' Salam ');

Où met-on les codes JavaScript?

- Vous pouvez inclure JavaScripts dans :

① l'élément (`<script>...</script>`),

Ex: - `<script> alert(' Salam ');</script>`
- `<script src="myScript.js"></script>`

② un événement d'un objet de la page

Ex: `onclick="alert(' Salam ')"`



`alert(' Salam ');`

```
1 <!doctype html>
2 <html>
3 <head>
4   <title> Salam World </title>
5   <script> alert(' Salam ');</script>
6 </head>
7 <body>
8   <a href="#" onclick="alert(' Salam ')">Cliquez</a>
9
10 </body>
11 </html>
```

①

②

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Où met-on les codes JavaScript?

- La balise `<script>` ne peut pas contenir à la fois l'attribut `src` et le code.
 - (`<script>...</script>`),

Ex :

```
<script src="myScript.js">
    alert(' Salam ');
</script>
```



L'exemple ci-dessus peut être divisé en deux scripts pour fonctionner:

```
<script src="myScript.js"></script>
<script>
    alert(' Salam ');
</script>
```

La Syntaxe de JavaScript .

Règles générales

- Le langage JavaScript est sensible à la casse.



Var ≠ var ≠ VAR ≠ vAR ≠ VaR

- Pour écrire plusieurs instructions sur une même ligne, il est nécessaire de les séparer avec un point-virgule (;). (;).



Commande1; Commande2; Commande3;

- JavaScript considère que chaque nouvelle ligne est une instruction différente



Commande1
Commande2
Commande3
.....

La Syntaxe de JavaScript .

- Les lignes d'instructions sont regroupées en blocs d'instructions délimités par des accolades



{ ... instructions ... }

Les commentaires :

- Il existe deux méthodes permettant d'intégrer des commentaires à vos scripts.

- Sur une ligne : **// ... commentaire ligne...**

- Sur plusieurs lignes : **/* */**

commentaire multi-lignes

..... */

TP1 Exercice 1

a) Ecrire une page HTML contenant un script permettant d'afficher

« سلام للجميع »

b) Améliorer le script pour qu'il permette de demander le Prénom de l'utilisateur (méthode `prompt()`) et afficher

« سلام ' 'إسمك' ' »

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Comment afficher les données en JavaScript?

JavaScript peut afficher les données par différentes manières:

1. L'écriture dans un élément HTML, en utilisant **innerHTML** .
2. L'écriture dans la sortie HTML en utilisant **document.write ()** .
3. L'écriture dans une zone d'alerte, en utilisant **window.alert ()** .
4. L'écriture dans la console du navigateur, en utilisant **console.log ()** .



```
HTML 1 <!doctype html>
      2 <html>
      3 <head>
      4   <title> Salam World </title>
      5 </head>
      6 <body>
      7 <p id="demo"></p>
      8 <script>
      9 document.getElementById("demo").innerHTML = 'les données';
     10 document.write ('les données');
     11 window.alert('les données');
     12 console.log('les données');
     13 </script>
     14 </body>
     15 </html>
```

1
2
3
4

La méthode **getElementById ()** renvoie l'élément qui possède l'attribut **ID** avec la valeur spécifiée.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript ?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

La console JavaScript

- la console JavaScript. C'est un outil secret que les développeurs utilisent pour tester de courts programmes JavaScript.

Pour accéder à la console dans :	
les navigateurs	Raccourci clavier (Windows)
Google Chrome	Ctrl + Shift + J
Mozilla Firefox	Ctrl + Shift + K
Internet Explorer	F12
Opera	Ctrl + Shift + I

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Boîtes de messages

- Il existe 3 types de boîtes de messages peuvent être affichés en utilisant Javascript : Alerte, Confirmation et Invite

1. Méthode alert()

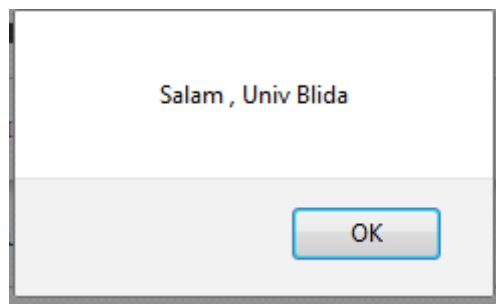
sert à afficher à l'utilisateur des informations simples de type texte. Une fois que ce dernier a lu le message, il doit cliquer sur OK pour faire disparaître la boîte.

2. Méthode confirm()

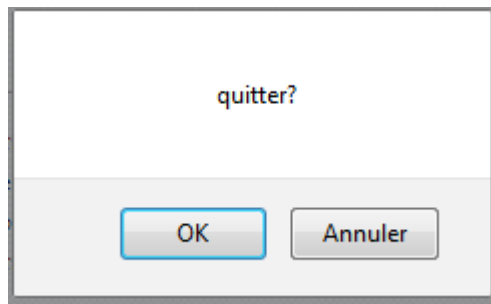
permet à l'utilisateur de choisir entre les boutons OK et Annuler.

3. Méthode prompt()

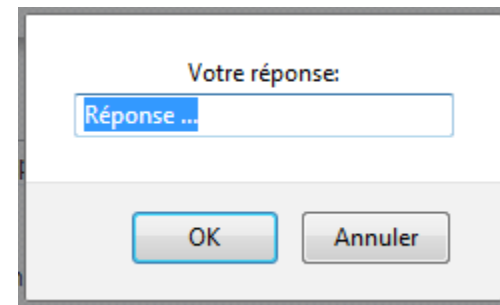
permet à l'utilisateur de taper son propre message en réponse à la question posée, Cette méthode retourne la chaîne que l'utilisateur a entrée



```
alert('Salam , Univ Blida')
```



```
confirm("quitter?")
```



```
prompt("Votre réponse?", "Réponse ..")
```

```
<html>
  <head>
    <title> Exercice1 </title>
  </head>
  <body>
    <script>
      document.write('سلام للجميع');
    </script>
  </body>
</html>
```

```
<html>
  <head>
    <title> Exercice1 </title>
  </head>
  <body>
    <script>
      alert('سلام للجميع');
    </script>
  </body>
</html>
```

```
<body>
  <p id="demo"></p>
  <script>
    document.getElementById("demo").innerHTML = 'سلام للجميع';
  </script>
</body>
</html>
```

Variables, Types de données & Les opérateurs

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les variables en JavaScript.

- En JavaScript, une variable peut être déclarée en utilisant «**let**» ou «**var**» .



var x,y,z;



let x,y,z;

- La portée de la déclaration «var» est **globale** indépendamment de son emplacement de déclaration.
- tandis que la variable déclarée à l'aide de l'instruction « let » a une portée de bloc ou une portée incluse. où un bloc peut être défini comme du code entre accolades { }..

La déclaration «let» a été introduite dans la version 6 de la norme ECMAScript souvent appelée ES6 ou ES 2015.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les variables en JavaScript.

- Les mot-clé **var** et **let** permet de déclarer une ou plusieurs variables.



```
let x,y,z;
```

- Il est possible de lui affecter une valeur par (=).



```
let x=12,y;
```

- Les variables en JavaScript n'ont pas de type défini.



```
var x='dz',y=1.5
```

- Nom de variable sensible à la casse.



```
var x,X,g,G;
```

- Il est possible d'affecter une valeur à plusieurs variables a la fois.



```
x = y = z = 40;
```

- Les noms de variables ne peuvent contenir que des lettres, chiffres,

et le caractère "_" et "\$".



```
myVariabl_1=5;
```

- Les noms doivent commencer par une lettre.



Upper Camel Case (Pascal Case):façon de joindre plusieurs mots en un nom de variable: **nomEtPrenom**

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les variables en JavaScript.



N'importe quelle **valeur** peut être stockée dans n'importe quelle **variable**. le JavaScript est un langage typé *dynamiquement*.



```
let c;  
    c= 2;  
c+=2; // c=4  
c*=2; // c=8  
c+='mot'; // c='8mot'
```

1. [Qu'est ce que JavaScript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les types de données en JavaScript

En JavaScript, il y 'a 3 types de base (**Types primitifs**):

1. Les nombres (Number):

Ils se classent en deux catégories :

☐ les nombres entiers .

☐ les nombres à virgule .  On n'utilise pas la virgule, mais le **point**



```
var x = 35;
```



```
let x = 35;
```



```
let pi = 3.14;
```

2. Les chaînes de caractères (String):

Une chaîne de caractère, c'est du texte.

Elle est délimitée par des guillemets " ou bien par des apostrophes '

 On utilise des guillemets " ou bien des apostrophes ' .



```
let nom = " Ahmed " ;
```



```
let nom = ' Ahmed ' ;
```

2. Les booléens (Boolean):

Ces variables ont deux valeurs possibles :

☐ vrai (**true**), ou

☐ faux (**false**).



```
let age = prompt("Quel age avez-vous ?");  
let est_majeur = (age >= 18);
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les types de données en JavaScript

JavaScript inclut des valeurs spéciales:

1. Null

qui signifie l'absence de données dans une variable, type et valeur inexistant, vides



```
let x = "";
```

2. Undefined

Une variable dont le contenu n'est pas clair car elle n'a jamais stocké de valeur , (variable non initialisée).



```
var nom;
```

3. Autres Valeurs spéciales

NaN : (Not a number).

Infinity : Nombre infini



```
var x = 0 / 0;
```



```
var x = 8 / 0;
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les opérateurs dans JavaScript.

En JavaScript, on peut faire les opérations et l'affectation avec les opérateurs:

1. Opérateurs arithmétiques

Les opérateurs arithmétiques sont utilisés pour effectuer l'arithmétique sur les nombres:

Opérateur	Description	Exemple(X=6 , Y=3)	Resultat
+	Addition	X = Y + 5;	X = 8;
-	Soustraction	X = X - Y;	X = 3;
*	Multiplication	X = X * Y;	X = 18;
/	Division	X = X / Y;	X = 2;
%	Modulos (les reste)	X = X % Y; Y = 10 % 3;	X = 0; Y = 1;
++	Incrémententation	X ++	X = 7;
--	Décrémententation	X --	X = 5;

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript ?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les opérateurs dans JavaScript.

En JavaScript, on peut faire les opérations et l'affectation avec les opérateurs:

2. Opérateurs d'affectation

Les opérateurs d'affectation attribuent des valeurs aux variables:

Opérateur	Exemple	Même que
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y



Il y'a aussi : <<= ; >>= ; >>>= ; &= ; ^= ; |=

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les opérateurs dans JavaScript.

En JavaScript, on peut faire les opérations et l'affectation avec les opérateurs:

3. Opérateurs de comparaison

comparent deux variables ,le résultat est **true** ou **false**..

Opérateur	Description	Exemple(X=6 , Y=3)	Résultat
==	égal à	X == Y	true
===	valeur égale et type égal	X === '6'	false
!=	pas égal	X != Y	true
!==	valeur non égale ou de type différent	X !== '6'	true
>	plus grand que	X > Y	true
<	Inférieur à	X < Y	false
>=	supérieur ou égal à	X >= 6	true
<=	inférieur ou égal à	X <= 6	true
?	Opérateur conditionnel (ternaire)	Result=(X < Y) ? 25:33	33

 **variablename = (condition) ? value1:value2**

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les opérateurs dans JavaScript.

En JavaScript, on peut faire les opérations et l'affectation avec les opérateurs:

4. Opérateurs logiques

Opérateur	Description	Exemple (X=6 , Y=3)	Résultat
&&	Et logique (and)	(x < 10 && y > 1)	true
 	Ou logique (or)	(x == 5 y == 5)	false
!	Non logique (not)	!(x == y)	true

4. Opérateurs de type

Operator	Description	Exemple (X=6)	Résultat
typeof	Renvoie le type d'une variable	typeof X	"number"
instanceof	Renvoie true si un objet est une instance d'un type d'objet	X instanceof Array;	false

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les opérateurs dans JavaScript.

En JavaScript, on peut faire les opérations et l'affectation avec les opérateurs:

5. Opérateurs bit à bit (Bitwise)

Les opérateurs de bits fonctionnent sur des nombres de 32 bits .:

Opérateur	Description	Exemple	Même que	Résultat	Décimal
&	Et (AND)	5 & 1	0101 & 0001	1	1
 	Ou (OR)	5 1	0101 0001	101	5
~	Non (NOT)	~ 5	~ 0101	1010	10
^	(XOR)	5 ^ 1	0101 ^ 0001	100	4
<<	Décalage binaire à gauche	5 << 1	0101 << 1	1010	10
>>	Signed right shift	5 >> 1	0101 >> 1	10	2
>>>	Zero fill right shift	5 >>> 1	0101 >>> 1	10	2



Tout opérande numérique dans l'opération est convertie en un nombre de 32 bits. Le résultat est reconverti en un numéro JavaScript.

4

Les instructions conditionnelles & Les boucles

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les instructions conditionnelles

Les instructions conditionnelles sont utilisées pour effectuer différentes actions en fonction de différentes conditions.

1. if... else



```
if (condition) {  
    bloc de code à exécuter si la condition est vraie (true)  
} else {  
    bloc de code à exécuter si la condition est fausse (false)  
}
```



Utilisez l'instruction **else if** pour spécifier une nouvelle condition si la première condition est fausse.



```
var time = prompt('quelle heure est-il?(sans minutes)');  
if (time < 10) {  
    greeting = "Good morning";  
} else if (time < 20) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les instructions conditionnelles

2. switch ... case



```
switch (expression) {  
  case Valeur1:  
    bloc de code à exécuter  
    break;  
  ...  
  case Valeurn:  
    bloc de code à exécuter  
    break;  
  default:  
    bloc de code à exécuter  
}
```

default : Une clause exécutée si aucune correspondance n'est trouvée avec les clause case (et/ou s'il n'y a pas de break pour les clauses case précédentes).

break : Si on omet une instruction break, le script exécutera les instructions pour le cas correspondant et aussi celles pour les cas suivants jusqu'à la fin de l'instruction switch ou jusqu'à une instruction break.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript ?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les instructions conditionnelles

2. switch ... Case (Exemples)



```
switch (new Date().getDay()) {  
  case 0:  
    day = "Sunday";  
    break;  
  case 1:  
    day = "Monday";  
    break;  
  case 2:  
    day = "Tuesday";  
    break;  
  case 3:  
    day = "Wednesday";  
    break;  
  case 4:  
    day = "Thursday";  
    break;  
  case 5:  
    day = "Friday";  
    break;  
  case 6:  
    day = "Saturday";  
}
```



La méthode `getDay ()` renvoie le jour de la semaine sous la forme d'un nombre compris entre 0 et 6.

```
var expr = prompt('Combien ça coûte?(Tapez le nom du fruit)');  
switch (expr) {  
  case "Oranges":  
    alert("Oranges : 120 dz le kilo.");  
    break;  
  case "Pommes":  
    alert("Pommes : 150 dz le kilo.");  
    break;  
  case "Bananes":  
    alert("Bananes : 300 dz le kilo.");  
    break;  
  case "Cerises":  
    alert("Cerises : 250 dz le kilo.");  
    break;  
  case "Mangues":  
  case "Fraise":  
    alert( expr + ": 420 dz le kilo.");  
    break;  
  default:  
    alert("Désolé, nous n'avons plus de " + expr + ".");  
}
```

- | | |
|--|---|
| 1. Qu'est ce que Javascript? | 8. Les types de données en JavaScript |
| 2. Que peut-on faire avec JavaScript ? | 9. Les opérateurs dans JavaScript. |
| 3. Où met-on les codes JavaScript? | 10. Les fonctions dans JavaScript . |
| 4. Comment afficher les données ? | 11. Les instructions conditionnelles |
| 5. Boîtes de messages | 12. Les boucles dans JavaScript . |
| 6. La Syntaxe de JavaScript . | 13. Les objets dans JavaScript . |
| 7. Les variables en JavaScript. | 14. Les événements. |

Les boucles dans JavaScript .

JavaScript prend en charge différents types de boucles:

1. Boucle : for

2. Boucle : for / in

3. Boucle : while

4. Boucle : do / while

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les boucles dans JavaScript ?

1. Boucle : for



```
for (initialisation ; condition ; incrémentation)
{
    instructions
}
```



Les parties Initialisation et incrémentation d'une boucle for peuvent contenir plusieurs instructions.



```
for (let i=1; i<=10; i++)
    somme += i;
```



```
for (let i = 0, j=5; i < 5; i++, j++) {
    text = "The number is " + (i+j) + "<br>";
    document.write(text);
}
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les boucles dans JavaScript ?

2. Boucle : for ... in



```
for (variable in objet) {  
    Instruction 1  
    InstructionN  
}
```



L'instruction for .. in boucle à travers les propriétés d'un objet:



```
var obj = {a:1, b:2, c:3};  
  
for (var prop in obj) {  
    document.write (`obj.${prop} = ${obj[prop]}`);  
}
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les boucles dans JavaScript ?

3. Boucle : while



```
while (condition)
{
    Instruction 1
    InstructionN
}
```

 La condition est évaluée avant d'exécuter l'instruction contenue dans la boucle.



```
while (i < 10) {
    text += "The number is " + i;
    i++;
}
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les boucles dans JavaScript ?

4. Boucle : do .. while



do

{

Instruction 1

InstructionN

}

while (condition)



Cette boucle exécutera le bloc de code une fois, avant de vérifier si la condition est vraie, alors elle répétera la boucle tant que la condition est vraie.

do{

text += "The number is " + i;

i++;

}

while (i < 10)

Les fonctions

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les fonctions dans JavaScript ?

En JavaScript, on peut faire les opérations et l'affectation avec les opérateurs:

Une fonction est un ensemble d'instructions que l'on peut appeler à l'aide de son nom.



```
function Nom_Fonction (Arg1, ..., ArgN) {  
    Instruction 1  
    .  
    .  
    InstructionM  
    return valeur  
}
```



La présence des **arguments** (paramètres) et de sortie **return** dans la fonction ne sont **pas toujours (pas obligatoire)**.



- Les noms de fonctions suivent les mêmes règles que les noms de variables.
- L'appel d'une fonction doit toujours comporter les parenthèses, même si la fonction ne prend pas d'argument.
- on déclare généralement les fonctions dans l'en-tête (<HEAD> ... </HEAD>) afin qu'elles puissent être appelées de n'importe où dans la page.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript ?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les fonctions dans JavaScript ?

Règles de paramètres

- Les définitions de fonction JavaScript ne spécifient pas les **types de données** pour les paramètres.
- Les fonctions JavaScript **n'effectuent pas de vérification** de type sur les arguments transmis.
- Les fonctions JavaScript **ne vérifient pas le nombre** d'arguments reçus.
- Si une fonction est appelée avec des arguments manquants (moins que déclaré), les valeurs manquantes sont définies comme : **undefined**

L'objet (Arguments)

- L'objet argument contient **un tableau** des arguments utilisés lorsque la fonction a été appelée



```
x = sumAll(8, 22, 36, 912, 78, 2);
```

```
function sumAll() {  
    var i;  
    var sum = 0;  
    for (i = 0; i < arguments.length; i++) {  
        sum += arguments[i];  
    }  
    return sum;  
}
```

la fonction est appelée avec 6 arguments

sumAll(): la fonction est créée sans arguments

arguments.length : donne le nombre des argument

arguments[i]: donne l'argument N° i, ex: N° 4 est 78

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les fonctions dans JavaScript ?

Les fonctions anonymes

Il n'est pas obligatoire en JavaScript de donner un nom à une fonction au moment de la déclaration.



```
var nomDeVariable = function (Arg1, ..., ArgN) {
```

```
    Instruction 1
```

```
    ..
```

```
    InstructionM
```

```
    return valeur
```

```
}; // Attention ici il faut le point virgule.
```

```
// Appel avec nom de variable + parenthèses
```

```
nomDeVariable(....);
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les fonctions dans JavaScript ?

Les fonctions fléchées

Les fonctions fléchées (arrow functions) ont été introduites avec ES6 (ES2015), Les fonctions fléchées sont des fonctions qui possèdent une syntaxe très compacte, ce qui les rend très rapide à écrire. Les fonctions fléchées utilisent le signe **=>** qui leur a donné leur nom à cause de sa forme de flèche.



(parametres) => { instructions }



```
let somme = (a, b) => a + b;
```

```
alert(somme(1, 2)); // Appel
```

```
let double = n => n * 2;
```

```
alert(double(3));
```

```
const helloES6 = name => `Hello ${name}`
```

```
console.log(helloES6('Ahmed')) // Hello Ahmed
```

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les fonctions dans JavaScript ?

Les fonctions récursives

Une fonction récursive est une fonction dans laquelle figure un appel à elle même.



```
function exempleRecursive (Arg1, ..., ArgN) {  
    Instruction 1  
    ..  
    InstructionM  
    exempleRecursive(Arg1, ..., ArgN); // Appel récursif  
}  
  
exempleRecursive(Arg1, ..., ArgN); // Premier appel
```



Dans une fonction récursif, il faut toujours prévoir une **condition de sortie** qui cessera son exécution. Sinon, la fonction se rappellera **indéfiniment**.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les fonctions dans JavaScript ?

Les fonctions récursives

EXEMPLE



```
function factorielle(n) {  
    alert('Calcul factorielle : '+n);  
    if (n == 0) return 1; // condition de sortie  
    else return n*factorielle(n-1); // Appel récursif  
}  
  
alert( factorielle (5));
```

6

Les objets

- | | |
|--|---|
| 1. Qu'est ce que Javascript? | 8. Les types de données en JavaScript |
| 2. Que peut-on faire avec JavaScript ? | 9. Les opérateurs dans JavaScript. |
| 3. Où met-on les codes JavaScript? | 10. Les fonctions dans JavaScript . |
| 4. Comment afficher les données ? | 11. Les instructions conditionnelles |
| 5. Boîtes de messages | 12. Les boucles dans JavaScript . |
| 6. La Syntaxe de JavaScript . | 13. Les objets dans JavaScript . |
| 7. Les variables en JavaScript. | 14. Les événements. |

Les objets dans JavaScript .

Un objet est un élément nommé ayant des

- 1. Propriétés** : paramètres que vous vérifier et modifier.
- 2. Méthodes** : actions que l'objet est capable d'effectuer.
- 3. Événements** : choses qui arrivent à l'objet, auxquelles celui-ci peut répondre automatiquement par une action.

- | | |
|--|---|
| 1. Qu'est ce que Javascript? | 8. Les types de données en JavaScript |
| 2. Que peut-on faire avec JavaScript ? | 9. Les opérateurs dans JavaScript. |
| 3. Où met-on les codes JavaScript? | 10. Les fonctions dans JavaScript . |
| 4. Comment afficher les données ? | 11. Les instructions conditionnelles |
| 5. Boîtes de messages | 12. Les boucles dans JavaScript . |
| 6. La Syntaxe de JavaScript . | 13. Les objets dans JavaScript . |
| 7. Les variables en JavaScript. | 14. Les événements. |

Les objets dans JavaScript ?

Outre les objets natifs et prédéfinis, nous pouvons également créer nos propres objets en JavaScript.

Nous disposons de **trois** façons pour **créer** de nouveaux objets :

1. On peut utiliser le mot clef ***new*** et créer un objet à partir de ***Object()*** ;
2. On peut créer un **objet littéral** ;
3. On peut définir un constructeur puis créer des objets à partir de celui-ci
(***comme une fonction***).

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les objets dans JavaScript ?

1. Création simple d'un objet : (`var objet1 = new Object ;`)



```
var nouvel_obj = new Object();  
nouvel_obj.pram1 = valeur1;  
.  
.  
nouvel_obj.pramN = valeurN;  
nouvel_obj.methode1 = function(){ les instructions };  
.  
.  
nouvel_obj.methodeN = function(){ les instructions };
```



Notez qu'il est aujourd'hui généralement **déconseillé** de créer un objet de cette façon pour des raisons de performance du code. Cette méthode ne vous est donc présentée qu'à titre d'exemple.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les objets dans JavaScript ?

1. Création simple d'un objet : (`var objet1 = new Object ;`)

Exemple



<script>

//On crée un objet en utilisant new Object()

var etudiant = **new Object()**;

//On définit des propriétés pour notre objet

etudiant.prenom = "Hamid";

etudiant.nom = "Aissa";

etudiant.gendre = "masculin";

etudiant.age = 22;

//On définit des méthodes pour notre objet

etudiant.identite = **function()** {

return alert(**this**.nom + " " + **this**.prenom)

};

//On utilise par exemple la méthode identite

etudiant.identite();

</script>

l'utilisation du mot clef **this**
fait référence à l'objet
couramment utilisé.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les objets dans JavaScript ?

2. Création d'un objet littéral : (`var objet1 = {.....}`)



```
var nouvel_obj = {  
  pram1 : valeur1;  
  .  
  .  
  pramN: valeurN;  
  Methode1: function(){ les instructions };  
  .  
  .  
  methodeN : function(){ les instructions };  
}
```



Créer un objet littéral est la manière généralement recommandée de créer un objet, et c'est également la manière la plus simple de procéder.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript ?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les objets dans JavaScript ?

2. Création d'un objet littéral : (`var objet1 = {...,...,...}`)

Exemple



`<script>`

//On crée un objet

`var etudiant = {`

//On définit des propriétés pour notre objet

`prenom : "Hamid",`

`nom : "Aissa",`

`genre : "masculin",`

`age : 22,`

//On définit des méthodes pour notre objet

`identite : function() {`

`return alert(this.nom + " " + this.prenom)`

`}`

`}`

//On utilise par exemple la méthode identite

`etudiant.identite();`

`</script>`

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les objets dans JavaScript ?

3. Création des objets à partir d'un constructeur: (`function Constr1(...,...){}`)

La dernière méthode pour créer des objets en JavaScript est de créer un constructeur puis ensuite de créer des objets à partir de celui-ci.



```
function Nom_Constructeur(val1,...,valN)= {  
    pram1 = val1;  
    .  
    .  
    pramN = valN;  
    Methode1 = function() { les instructions };  
    .  
    .  
    methodeM = function() { les instructions }  
}
```

1st : créer un constructeur

2nd : créer des objets

```
var Objet1 = new Nom_Constructeur (Obj1_val1,...,Obj1_valN);  
var Objet2 = new Nom_Constructeur (Obj2_val1,...,Obj2_valN);
```



Nous utiliserons cette façon de faire pour des gros projets pour lesquels nous aurons à créer beaucoup d'objets similaires.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les objets dans JavaScript ?

3. Création des objets à partir d'un constructeur (`function Constr1(){}`) Exemple



`<script>`

```
//On crée un constructeur
function etudiant(p, n, g, a){
    //On définit les propriétés
    this.prenom = p;
    this.nom = n;
    this.gendre = g;
    this.age = a;
    this.identite = function() {
        return alert(this.nom +" "+ this.prenom)
    }
}

//On crée des objet
var ahmed = new etudiant("Ahmed","Aissa","masculin",22);
var salima = new etudiant("Salima","moslim","feminin",20);
//On affiche par exemple la méthode identite
ahmed.identite();
salima.identite();
```

`</script>`

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les objets dans JavaScript ?

Accès aux propriétés et méthodes de l'objet

Vous pouvez accéder aux propriétés de l'objet de deux manières:

1 `objectName.propertyName`

2 `objectName["propertyName"]`

Vous accédez à une méthode d'objet avec la syntaxe suivante:

`objectName.methodName()`

1. Qu'est ce que Javascript?	8. Les types de données en JavaScript
2. Que peut-on faire avec JavaScript ?	9. Les opérateurs dans JavaScript.
3. Où met-on les codes JavaScript?	10. Les fonctions dans JavaScript .
4. Comment afficher les données ?	11. Les instructions conditionnelles
5. Boîtes de messages	12. Les boucles dans JavaScript .
6. La Syntaxe de JavaScript .	13. Les objets dans JavaScript .
7. Les variables en JavaScript.	14. Les événements.

Les évènements

- Un événement en javascript est une fonction (native) destinées aux objets que la page peut contenir (boutons, images, liens, formulaires, ...).
- Ces fonctions ne s'activent qu'à la suite d'une action bien précise de l'utilisateur (un clic, une touche du clavier, un mouvement de souris, un changement de taille de fenêtre, ...) .
- Ces événements se placent généralement dans les balises HTML elles même et lorsqu'ils sont activés, appellent une fonction.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les évènements

Voici donc les 23 événements de JavaScript par ordre alphabétique :

Événement	Objet(s) affecté(s)	Description
onAbort (annuellement)	Image	Se produit lorsque l'utilisateur annule le chargement d'une image (clic sur un lien ou bouton "stop" du navigateur).
onBlur (sortie)	Button, Checkbox, FileUpload, Layer, Password, Radio, Reset, Select, Submit, Text, Textarea et window	Se produit lorsque l'utilisateur sort d'un champ de formulaire HTML, d'un cadre ou d'une fenêtre (window), en appuyant sur la touche TAB du clavier ou en utilisant la souris.
onChange (changement)	FileUpload, Select, Text et Textarea	Se produit lorsque l'utilisateur modifie la valeur d'une boîte liste, d'une boîte à liste déroulante, d'une boîte texte
onClick (clic)	Button, Checkbox, document, Link, Radio, Reset et Submit	Se produit lorsque l'utilisateur enfonce puis relâche le bouton de la souris sur un bouton, une option dans un formulaire, un lien ou sur le document (en gros un clic).
onDbClick (double clic)	Area, document et Link	Se produit lorsque le bouton de la souris est enfoncé deux fois rapidement puis relâché sur un élément de formulaire ou un lien
onDragDrop (glisser déposer)	window	Se produit lorsque l'utilisateur dépose un objet (un fichier par exemple) dans la fenêtre du navigateur Web à l'aide de la souris.
onError (erreur)	Image et window	Se produit lorsqu'il y a une erreur au cours du chargement d'une image ou d'un document.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les évènements

Voici donc les 23 événements de JavaScript par ordre alphabétique :

Événement	Objet(s) affecté(s)	Description
onFocus (focus)	Button, Checkbox, FileUpload, Layer, Password, Radio, Reset, Select, Submit, Text, Textarea et window	Se produit lorsqu'un élément (champ de formulaire, cadre, layer, etc.) ou un objet (window) obtient le focus. Le focus peut être obtenu par l'intermédiaire de la souris, de la touche TAB du clavier ou par une méthode de focus.
onKeyDown (enfoncement de touche)	document, Image, Link et Textarea	Se produit au moment où l'utilisateur enfonce une touche du clavier.
onKeyPress (enfoncement de touche)	document, Image, Link et Textarea	Se produit au moment où l'utilisateur relâche la touche qu'il venait d'enfoncer.
onKeyUp (relâchement de touche)	document, Image, Link et Textarea	En principe, l'événement onKeyUp se produit au moment où l'utilisateur relâche une touche du clavier.
onLoad (chargement)	Image, Layer et window	Se produit lorsque le navigateur a terminé le chargement d'une page HTML, de tous les cadres (FRAMES) de l'élément ou d'une image.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript .](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les évènements

Voici donc les 23 événements de JavaScript par ordre alphabétique :

Événement	Objet(s) affecté(s)	Description
onMouseDown (clic)	Button, document et Link	Se produit lorsque l'utilisateur enfonce un des boutons de la souris (tout pareil que le clic).
onMouseMove (déplacement su pointeur)		Se produit lorsque le pointeur de la souris est déplacé et passe sur l'objet.
onMouseOut (sortie de pointeur)	Area, Layer et Link	Se produit à chaque fois que l'utilisateur déplace le pointeur de la souris hors d'une zone contenant un lien (images en coordonnées, lien).
onMouseOver (pointeur au-dessus)	Area, Layer et Link	Se produit à chaque fois que le pointeur de la souris entre dans une zone contenant un lien (lien ou images en coordonnées).
onMouseUp (relâchement clic)	Button, document et Link	Se produit lorsque l'utilisateur relâche le bouton de la souris.
onMove (déplacement)	window	Se produit lorsque l'utilisateur ou un script déplace une fenêtre ou un cadre..

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les évènements

Voici donc les 23 événements de JavaScript par ordre alphabétique :

Événement	Objet(s) affecté(s)	Description
onReset (effacement)	Form	Se produit lorsque l'utilisateur active le bouton de type reset d'un formulaire.
onResize (retailage)	window	Se produit lorsque l'utilisateur modifie les dimensions de la fenêtre du navigateur Web.
onSelect (sélection)	Text et Textarea	Se produit lorsque l'utilisateur sélectionne du texte dans une boîte texte ou dans une boîte texte multiligne.
onSubmit (validation)	Form	Se produit lorsque l'utilisateur envoie les données d'un formulaire. Par exemple, la fonction appelée peut vérifier la validité des données saisies dans le formulaire et empêcher l'envoi de ce dernier si certaines informations s'avèrent non valides.
onUnload (déchargement)	window	Se produit au moment où le document HTML courant est retiré de la fenêtre au profit d'un autre ou lorsque le logiciel de navigation est fermé par l'utilisateur.

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)
14. [Les événements.](#)

Les évènements

éléments	attribut	événements
zone active (lien, ligne de tableau) bouton	onmouseover	début de survol par le pointeur
	onmousemove	survol avec déplacement
	onmouseout	fin du survol
	onmousedown	bouton du pointeur enfoncé
	onmouseup	bouton du pointeur relâché
	onclick	clic
zone de saisie ou lien	onfocus	l'élément devient la cible du focus (l'entrée au clavier lui est envoyée)
	onblur	perte du focus
case de texte	onchange	changement du contenu (et perte du focus)
	onkeypress	saisie au clavier
liste ou menu	onchange	sélection d'un des éléments
formulaire	onsubmit	données du formulaire sur le point d'être envoyées
	onreset	données sur le point d'être remises à zéro
document	onload	vient d'être chargé
	onunload	s'apprête à être déchargé (suivi de lien ou fermeture de la fenêtre)

1. [Qu'est ce que Javascript?](#)
2. [Que peut-on faire avec JavaScript ?](#)
3. [Où met-on les codes JavaScript?](#)
4. [Comment afficher les données ?](#)
5. [Boîtes de messages](#)
6. [La Syntaxe de JavaScript .](#)
7. [Les variables en JavaScript.](#)
8. [Les types de données en JavaScript](#)
9. [Les opérateurs dans JavaScript.](#)
10. [Les fonctions dans JavaScript .](#)
11. [Les instructions conditionnelles](#)
12. [Les boucles dans JavaScript .](#)
13. [Les objets dans JavaScript .](#)